

TaskDrop

A reverse marketplace for local work. Someone posts what they need with a fair price attached; taskers nearby quote against it; the money sits in escrow until the poster says the work is done. This document is the specification of every flow in the product, written against the code and the database that exist today — not against an intention.

Market India · INR **Platforms** Android, Web
Stack Expo SDK 57 · React Native 0.86 · Supabase **Payments** Razorpay (test mode)
Status Feature-complete, pre-launch

CONTENTS

The premise	The flows
People & roles	Permission model
Product principles	Running it
The money model	Known gaps
Task state machine	Acceptance checks

01 The premise

Most marketplaces ask the worker to advertise and the buyer to shop. TaskDrop inverts it.

A **poster** publishes an open request — what needs doing, where, by when, and a **benchmark price** they think it is worth. That benchmark is published, not hidden. Nearby **workers** see the request and send a **quote**: their price and how long they need. The poster compares the quotes that arrive and **locks** one, which assigns the job and creates an escrow obligation. The poster pays that escrow before any work is allowed to start. When the worker marks the job done and the poster confirms, the money is released to the worker — minus commission — and clears into a withdrawable balance a week later.

The benchmark price is the product. It stops the race to the bottom that open bidding produces, because the poster has already said out loud what the job is worth before anyone competes for it.

Three kinds of request

PILLAR	MEANS	EXAMPLE
services	Someone does a job for you	Rewire two bedroom sockets
procurement	Someone buys or fetches something	Collect a part from a dealer in Peenya
local_intel	Someone finds something out on the ground	Check whether that flat is really vacant

02 People & roles

One account, two modes. Nobody signs up as "a worker" or "a poster" — every profile holds both roles and a toggle decides which side of the app they are looking at.

ROLE	WHAT THEY DO	HOW IT IS HELD
poster	Publishes requests, compares quotes, funds escrow, confirms completion, rates the worker.	Granted on signup.
worker	Browses requests, sends quotes, starts and submits work, withdraws earnings.	Granted on signup.
admin	Marks payouts paid, settles disputes, runs the clearing sweep.	Granted manually. No screen in the app — see §08.

The mode toggle

Mode is a view preference, not a permission. Switching to **Post a Request** changes the home feed, the Orders tab ("My requests" vs "My jobs"), the wallet framing (spending vs earnings) and the wording throughout. It does not change what the database will allow — that is decided per row, per action, by the rules in §07.

RULE — THE TWO SIDES NEVER MEET ON ONE TASK

- A poster cannot quote on his own request, in any mode. The database refuses it and the app no longer offers it: his own requests are excluded from browse and search, and opening one takes him to the poster side of that task — the quotes it has received.
- A worker cannot lock his own quote. Only the poster of the task can lock.

03 Product principles

Four rules that decided most of the arguments during the build. They are worth stating because breaking any of them is how this product would fail with real money in it.

1 · No work before payment

Fiverr's oldest rule, and the one this product got wrong first. A task cannot be started, submitted, or released unless `funded_at` is set — meaning a settled payment covering the full escrow exists against that exact task. Before this gate was added, tasks ran to completion and released money that had never been collected.

2 · The server decides, always

Every transition that touches money or state is a `SECURITY DEFINER` function that re-reads `auth.uid()` and re-checks the rules itself. The app cannot move money by writing a row. It asks, and the database says yes or no. A client is a rendering of state, never a source of it.

3 · Money is an integer count of paise

Never a float, never rupees. `price_minor`, `escrow_minor`, `balance_minor`, `clearing_minor`. Rupees exist only in `formatINR()`, at the last moment before a human reads them.

4 · Nothing invented is ever shown

If the real answer is zero, the screen says zero. A wallet with no data shows ₹0, not a plausible balance; a request with no quotes says "No quotes yet", not four imaginary taskers with ratings. Placeholder data that is indistinguishable from real data is the single most dangerous thing a money app can render, and every instance found during the build was removed.

04 The money model

All rates live in a `settings` table and are read at transaction time, so changing one does not require a deploy — and does not retroactively change a task already priced.

SETTING	VALUE	APPLIES
<code>worker_commission_pct</code>	20%	Taken off the worker's quote at release.
<code>poster_service_fee_pct</code>	3%	Added to the quote at lock, to form the escrow.
<code>post_start_cancel_penalty_pct</code>	5%	Paid to the worker if the poster cancels after work started.
<code>clearing_period_days</code>	7 days	Earnings sit in clearing before becoming withdrawable.
<code>review_window_days</code>	3 days	Window in which a rating can be left.
<code>max_video_seconds</code>	60 s	Cap on an attached video.

Worked example

A poster publishes a request with a benchmark of ₹1,200. A worker quotes ₹1,000 and the poster locks that quote.

Worker's quote (<code>locked_minor</code>)	₹1,000.00
Poster service fee, 3%	+ ₹30.00
Poster pays into escrow (<code>escrow_minor</code>)	₹1,030.00
Worker's quote	₹1,000.00
Commission, 20%	– ₹200.00
Worker receives	₹800.00

TaskDrop keeps ₹230 on a ₹1,000 job — 23% of the quote, 22.3% of what the poster paid. The poster sees the 3% as a line item; the worker sees the 20% before sending the quote. Neither number is hidden.

Where the money actually sits

- 1 The poster pays Razorpay. The funds land in **TaskDrop's Razorpay settlement account** — a normal merchant account, not a regulated escrow account. This is the honest description and it matters legally.

- 2 A webhook (`payment_link.paid`) marks the `payments` row `paid` , and `fund_task()` stamps `funded_at` on the task. The escrow is now recognised.
- 3 On release, the worker's net is credited to `wallets.clearing_minor` — an internal ledger entry, not a transfer. No money has left TaskDrop.
- 4 After the clearing period, `settle_cleared_earnings()` moves it to `balance_minor` , which is withdrawable.
- 5 A withdrawal debits `balance_minor` immediately and creates a `payouts` row. An operator sends the money by UPI or bank transfer and marks it `paid` .
- 6 If the task is cancelled instead, the escrow goes back the way it came — a Razorpay refund against the original payment, for the escrow less whatever the worker was already paid out of it.

Until an operator sends that transfer, every worker balance on the platform is a liability TaskDrop owes out of a single pooled account. That is a business risk and a compliance question, not a bug — but it should be understood before real money is taken.

05 Task state machine

Ten states. Every transition is a named function; nothing changes status by an ordinary table write.

FROM	EVENT	TO	WHO	FUNCTION
—	Request published	OPEN	poster	insert_tasks
OPEN	Quote locked	LOCKED	poster	lock_bid()
LOCKED	Escrow paid	LOCKED + funded_at	poster	fund_task()
LOCKED	Work started	TASK_STARTED	worker	start_task()
TASK_STARTED	Deadline passed	OVERDUE	system	— not automated
TASK_STARTED	Work submitted	WORK_DONE	worker	mark_work_done()
WORK_DONE	Changes asked for	REVISION_REQUESTED	poster	request_revision()
WORK_DONE	Poster confirms	COMPLETED	poster	confirm_release()
WORK_DONE	No response in window	AUTO_COMPLETED	system	— not automated
any live state	Dispute raised	DISPUTED	either party	open_dispute()
DISPUTED	Settled	COMPLETED / CANCELLED	admin	admin_resolve_dispute()
before submission	Cancelled	CANCELLED / OPEN	either party	cancel_task()

INVARIANTS

- `funded_at` is set exactly once and never cleared. A retry of `fund_task()` returns success without double-counting.
- Underpayment does not fund a task. Overpayment funds it and the difference is left for an operator to decide on, never guessed at.
- A worker cancelling before submission returns the task to `OPEN` and unlocks the quote — the poster keeps the request and the other quotes.
- Work already submitted cannot be cancelled by anyone. The only route out is a dispute.

06 The flows

Fifteen flows cover everything a person can do in the app. Each names its actor, what must already be true, the steps as the user experiences them, and what the server enforces regardless of what the screen offers.

F1 Sign up and identity

new user

phone OTP

- 1 Welcome screen states the premise in one line and offers **Get started**.
- 2 The user enters an Indian mobile number. An OTP is sent through the `phone-auth` edge function.
- 3 Entering the code creates the Supabase auth user. A trigger (`handle_new_user()`) creates the profile, grants both the `poster` and `worker` roles, and opens a zero-balance wallet.
- 4 Setup asks for a display name and a home location. Both are skippable; the display name defaults to `Tasker NNNN` from the phone number.
- 5 The user lands on the home feed in worker mode.

ENFORCED

- No password is ever collected. Phone possession is the only credential.
- Contact details stay masked between counterparties until a task starts.

F2 Post a request

poster

- 1 The **+ List a request** button opens the composer. The user picks a pillar (services, procurement, local intel).
- 2 Title and description. Optionally a photo or a video up to 60 seconds, captured or picked from the library, uploaded to a private bucket under an owner-scoped path.
- 3 Location: a map pin, then a Swiggy-style address form over it — house or flat, floor, landmark, and a label (Home / Work / Other). Only line one is required.
- 4 A deadline, expressed as a time limit in minutes, and a **benchmark price**.
- 5 Review and publish. The task is created **OPEN** and becomes visible to every worker except its own poster.

ENFORCED

- Media lives in a private bucket. Every read is a short-lived signed URL; there are no public object URLs.
- Publishing costs nothing. Money is not involved until a quote is locked.

F3 Discover work and send a quote

worker

- 1 The home feed lists open requests, newest first, each showing the poster, the pillar, the benchmark price and any media thumbnail. Sponsored listings (F13) sit at the top, marked.
- 2 Search narrows by text, pillar, price range, and a radius around a chosen point. The radius is applied to the list that is actually rendered, not merely reported.
- 3 Opening a request shows the full description, the media, the poster's record and rating, and a quote box pre-filled with the benchmark.
- 4 The worker adjusts the price and sends the quote. It appears on the poster's compare screen immediately.
- 5 The worker tracks it under **My jobs → Quoted**.

ENFORCED – THREE CONDITIONS, ALL REQUIRED

- The quote is filed under the sender's own id.
- The sender holds the worker role.
- The task is **OPEN** and was not posted by the sender.

WHEN A QUOTE IS REFUSED

- **Your own request** — cannot occur through the feed, which excludes it. Reaching the task by a shared link redirects to the poster side of it.
- **Already quoted** — the screen says so and offers *See my quote*. One quote per task is a unique index, not a UI check.
- **No longer open** — the screen says so and offers *Find another request*.

F4 Compare quotes and lock one

poster

- 1 **My requests** → **Open** lists the poster's live requests with a quote count.
- 2 Opening one shows the benchmark, the deadline, and every quote received — price, the worker's rating and job count, distance, and their promised completion time.
- 3 With no quotes yet the screen says so plainly and states that nothing is charged until a quote is locked.
- 4 Quotes sort by lowest price or by rating. Selecting one and confirming calls `lock_bid()`.
- 5 The task moves to **LOCKED**, an assignment is created holding `escrow_minor = quote + 3%`, and the poster is sent straight to payment (F5).

ENFORCED

- Only the task's poster can lock, and only while the task is **OPEN**.
- Locking is idempotent per worker — re-locking updates the escrow rather than creating a second assignment.
- Worker names and contacts stay masked until the task starts.

F5 Fund the escrow

poster

razorpay

- 1 The payment screen itemises the quote, the 3% fee and the total, then lists the methods — UPI, cards, netbanking, wallets — in the Zomato pattern.
- 2 Choosing a method creates a `payments` row (`purpose = 'escrow'` , carrying the task id) and opens the Razorpay payment link.
- 3 Razorpay's `payment_link.paid` webhook, signature-verified, marks the payment `paid` .
- 4 `fund_task()` checks the payment is the poster's own, settled, raised for escrow, raised for **this** task, and at least the escrow amount — then stamps `funded_at` .
- 5 The screen polls and moves on by itself. There is no "I've paid, check now" button; the app does not take the user's word for a payment.

ENFORCED

- A payment raised for one task can never fund another.
- A payment that has not settled cannot fund anything.
- Underpaying does not fund the task.
- The Razorpay key secret never reaches the app bundle. It is resolved server-side only.

F6 Start the work

worker

- 1 The locked job appears under **My jobs** → **To start**, showing the escrow as funded or awaiting payment.
- 2 A slide-to-start control calls `start_task()` .
- 3 The task moves to **TASK_STARTED** , the countdown to the deadline begins, and contact details unmask for both sides.

ENFORCED — THE FUNDING GATE

- `start_task()` refuses if `funded_at` is null. A worker cannot be put to work on money that was never collected, and the app tells them the poster has not paid yet rather than pretending otherwise.
- Only the assigned worker can start.

F7 Talk during the job

poster

worker

- 1 A thread exists per task and is reachable from the active-job screen on both sides.
- 2 Messages are text, delivered over Supabase realtime.
- 3 The thread stays readable after completion, as the record of what was agreed.

ENFORCED

- Only the poster and the assigned worker of that task can read or write the thread. No third party, including another worker who quoted, has access.

F8 Submit work, and ask for changes

worker

poster

- 1 The worker marks the job done — `mark_work_done()` — optionally attaching proof. The task becomes **WORK_DONE**.
- 2 The poster is prompted to confirm or to ask for changes.
- 3 `request_revision()` moves it to **REVISION_REQUESTED** with a note, and the worker can resubmit.
- 4 Escrow does not move in either direction during this loop.

ENFORCED

- Only the assigned worker can submit; only the poster can ask for changes.
- Submission requires a funded task.
- Once submitted, neither side can cancel — see F12.

F9 Release the money and rate

poster

escrow

- 1 The poster confirms. `confirm_release()` re-checks that the task is funded, was submitted, and has a live worker.
- 2 The worker's net — quote less 20% — is credited to `clearing_minor`.
- 3 The task becomes **COMPLETED** and `clear_at` is set to seven days out.
- 4 Both sides are invited to rate each other within three days. Ratings feed the record shown on every future quote.

ENFORCED

- Only the poster releases. There is no admin override on a healthy task.
- An unfunded task cannot be released: *"No escrow was ever paid for this task, so there is nothing to release."*

F10 Clearing and withdrawal

worker

payout

- 1 Released earnings sit in **clearing** for seven days. The wallet shows clearing and available as separate numbers, with the date each amount clears.
- 2 `settle_cleared_earnings()` moves due amounts into the available balance. It is safe to run repeatedly — a task settles at most once.
- 3 The worker chooses a payout destination: a UPI VPA or a bank account, saved and shown masked (`...8901`).
- 4 They enter an amount — typed or chosen from chips, capped at the available balance — and request it. `request_withdrawal()` debits the balance immediately and creates a `payouts` row at `requested`.
- 5 While still `requested`, the withdrawal can be cancelled or its destination changed; `cancel_withdrawal()` returns the money to the balance.
- 6 An operator transfers the money and marks the payout `paid`, or `failed` — which refunds the full amount to the wallet.

ENFORCED

- The balance is locked for update while checking, so two simultaneous requests cannot both pass.
- Withdrawing more than the available balance is refused. Clearing money is not available.
- A payout already `paid` or `failed` cannot be re-marked, so a refund cannot be issued twice.

F11 Cancellation

poster

worker

- 1 Either party can cancel any time before the work is submitted.
- 2 **Worker cancels:** the assignment is voided, the quote is unlocked and the task returns to **OPEN** with its other quotes intact. No penalty.
- 3 **Poster cancels before work started:** the task closes **CANCELLED**. No penalty.
- 4 **Poster cancels after work started:** 5% of the quote is credited to the worker's clearing as compensation for time already given, and the task closes.
- 5 Every cancellation is written to `cancellations_log` with who, when, at which phase, and what was paid or forfeited.

THE REFUND

- The moment `cancel_task()` succeeds, the app asks for the refund and Razorpay sends the money back to however the poster paid. Typically 5–7 working days at the bank's pace, not TaskDrop's.
- How much is worked out in Postgres by `escrow_refund_due()` — the escrow, less anything already refunded, less the 5% the worker was paid out of it. The caller cannot propose an amount.
- The refund is written down only after Razorpay has accepted it, and a payment can never be refunded past what was paid.
- Anything that slips through — the app closed mid-cancel, Razorpay unreachable, a task cancelled by a dispute ruling — appears in `refunds_outstanding` for an operator to clear. That view should normally be empty.

F12 Dispute

poster

worker

admin

- 1 Once work is submitted, cancelling is closed off. Either side can instead open a dispute with a reason.
- 2 The task moves to **DISPUTED** and the escrow freezes. Neither party can move it.
- 3 An operator reviews the thread and any attached proof.
- 4 `admin_resolve_dispute()` settles it one of two ways: **for the worker**, which releases exactly as a normal completion would, or **for the poster**, which cancels the task for refund.
- 5 The decision and its reason are written to `cancellations_log`.

F13 Promote a listing

poster

worker

revenue

- 1 Promote opens with a searchable picker of the user's own live listings — a request if they post, a service if they work.
- 2 They choose a daily budget and a number of days. An explainer states plainly what the budget buys: position at the top of the feed and in search, marked as sponsored.
- 3 `start_promotion()` creates the promotion and raises a payment.
- 4 On settlement, `activate_promotion()` makes it live. `cancel_promotion()` ends it early.
- 5 Sponsored ids are read by the feed and pinned to the top, always labelled.

ENFORCED

- A promotion only goes live against a settled payment.
- Only the owner of a listing can promote it.
- Sponsored placement is always visibly marked. It never masquerades as organic ranking.

F14 Wallet and money views

poster

worker

- 1 The wallet shows available and clearing separately, never as one blended number, with the date each clearing amount becomes withdrawable.
- 2 Top-up adds funds by the same payment path, with a ₹10 floor.
- 3 The money screen is named for the mode: **My spending and requests** for a poster, **My earnings and jobs** for a worker, and shows the figures that side actually cares about.
- 4 Every figure is scoped to the signed-in user by an explicit filter, never left to row security to narrow.

ENFORCED

- A wallet query that cannot be scoped returns nothing and the screen shows zero. It never falls back to a number that is not the user's.

F15 Operations

admin

- 1 Three jobs cannot be skipped: paying workers who have withdrawn, settling disputes, and running the clearing sweep.
- 2 There is deliberately **no admin screen in the app**. The functions exist and are run from the Supabase SQL editor, which authenticates as the project owner.
- 3 The procedure is written up in `docs/RUNNING_THE_BUSINESS.md`, with the exact statement for each job.

THE ONE THAT CANNOT WAIT

- Until the clearing sweep runs on a schedule, a worker's money stops at clearing and they cannot withdraw it. Enabling `pg_cron` and scheduling the nightly sweep is a launch blocker, not an improvement.

07 Permission model

Row-level security is on for every table. The app holds an anonymous key with no privileges of its own; what a person can see and do is decided per row by their `auth.uid()`.

How a policy reads

The insert policy on `bids` is the clearest example, and the one that produced the most visible bug in the build:

CONDITION	MEANS	WHAT THE USER IS TOLD
<code>worker_id = auth.uid()</code>	You can only file a quote as yourself	Never reached through the UI
<code>has_role(uid, 'worker')</code>	You hold the worker role	Never reached — every account has it
<code>task_open_for_bid(task, uid)</code>	Task is OPEN and not yours	"Your own request" → redirected · "No longer open" → explained

A policy failure is a correct answer to a wrong question. Postgres saying new row violates row-level security policy means the app offered something it should not have. The fix belongs in front of the button, not in the error handler — and the error handler still has to speak English, because there is always a race left over.

Other rules worth naming

- **Tasks** — open tasks readable by all; drafts and closed tasks by their poster and assigned worker only.
- **Bids** — visible to the worker who sent it and the poster of the task. Workers cannot see each other's quotes.
- **Messages** — the two parties to that task, nobody else.
- **Wallets, payouts, payments** — the owner only, plus admin.
- **Helper functions** live in a `private` schema that PostgREST does not serve, so they cannot be called from a client.
- **Secrets** resolve server-side in three tiers: environment, then Vault via a service-role-only function, then a compiled-in fallback for test keys. The anonymous key gets 401 from the secrets function — verified.

08 Running it

What has to be true for the app to work, beyond the code being correct.

JOB	CADENCE	HOW
Release cleared earnings	Nightly	<code>settle_cleared_earnings()</code> — needs <code>pg_cron</code>
Pay withdrawal requests	Daily	Transfer by UPI or bank, then <code>admin_mark_payout()</code>
Settle disputes	As raised	<code>admin_resolve_dispute()</code>
Refund cancelled escrow	Automatic	Sent on cancel; leftovers in <code>refunds_outstanding</code>
Check the numbers	Weekly	<code>platform_stats(30)</code>

Before taking real money

- 1 Enable UPI on the Razorpay account and register the `payment_link.paid` webhook.
- 2 Rotate the Razorpay keys — the test keys used during the build have been handled in plain text and must not survive into live mode.
- 3 Restrict the Google Maps key in Cloud Console to the Places API and the `com.taskdrop.app` package. It ships in the client bundle; that restriction is the only real protection it has.
- 4 Enable `pg_cron` and schedule the nightly clearing sweep.
- 5 Decide how escrow is held and say so in the terms. A pooled merchant account is not an escrow account, and workers' balances are unsecured liabilities against it.

09 Known gaps

Stated plainly, because a specification that only describes what works is not a specification.

BLOCKS

The clearing sweep is not scheduled

`pg_cron` is not enabled on the project. Until it is, released earnings sit in clearing forever and no worker can withdraw. One statement fixes it.

MANUAL

Refunds after a dispute ruling are still by hand

A poster cancelling their own task is refunded automatically. A dispute settled for the poster cancels the task from the operator's side, where no app is running to ask for the refund — so it lands in `refunds_outstanding` and has to be cleared from the Razorpay dashboard.

MANUAL

Payouts are hand-run

There is no Razorpay Payouts integration. A person reads the queue, sends UPI or a bank transfer, and marks the row. Workable at low volume, not past it.

MANUAL

OVERDUE and AUTO_COMPLETED never fire

Both states exist and the app renders them, but nothing transitions a task into either. A poster who goes quiet after work is submitted leaves the worker waiting indefinitely.

MANUAL

No admin surface

Removed by decision — the console showed queues that were usually empty and read as meaningless. Operations run through documented SQL instead. Correct for one operator; revisit at a second.

LATER

No notifications

Everything is pull. "You will be notified when the first quote arrives" is currently a promise the product cannot keep — either build push, or change the copy.

LATER

No identity verification

A phone number is the only thing known about anyone. Ratings accumulate from real completed jobs, which is honest but slow, and there is no KYC on either side.

LATER

iOS is not built

The codebase is one React Native app targeting Android and web. iOS needs a build, a developer account, and a review pass — no code changes expected.

10 Acceptance checks

The behaviours that must hold before this ships. Each is a thing to try, not a thing to assert.

#	CHECK	EXPECTED
A1	Post a request, then browse and search as the same account	Your own request appears nowhere in either
A2	Open your own request by its shared link, in worker mode	Lands on the quotes-received screen, not a quote box
A3	Open a request you have already quoted on	"You have already quoted on this task", with a way to see it
A4	Open a request with zero quotes as its poster	"No quotes yet" — no crash, no invented quotes
A5	Lock a quote, then try to start the job without paying	Refused, and the worker is told the poster has not paid
A6	Pay less than the escrow amount	Task is not funded
A7	Pay for task A, try to fund task B with it	Refused
A8	Release an unfunded task	Refused with the reason
A9	Withdraw more than the available balance	Refused; clearing money is not spendable
A10	Cancel a withdrawal, then check the wallet	Exact amount returned, once
A11	Call any money RPC with the anonymous key	Refused
A12	Read another user's wallet, bid, or message row	Empty, not an error — the row is simply not there
A13	Install the Android build and sign in on a real phone	Bundles and runs; web parity is not proof of this

Written against the codebase and the live database, September 2026. Where this document and the code disagree, the code is what ships — treat the difference as a bug in one of them and say which.